

Student Project Guide – House Price Prediction (End-to-End ML Web App)

Goal: Build a complete machine-learning product, from raw data to a deployed web app, and publish it on GitHub.

Rules:

- You may work **individually** or as a **team**.
- The **maximum team size is 3 students**.
- **Each one must submit the project individually** on the submission form, even if working in a team.
- Each student must provide the **GitHub repository** link in the submission form.
- Make sure the GitHub repository is accessible before submitting.

You will:

1. Download and explore the **House Price** dataset from Kaggle.
2. Build a **Jupyter notebook** that cleans the data, trains and evaluates a regression model, and **exports** it as a `.pkl` file.
3. Build a **FastAPI backend** that serves the model.
4. Build a **React frontend** where a user enters property details and sees the predicted price.
5. Publish everything to **GitHub** with a professional README.

A reference implementation of steps 3-5 already exists in this repository (`backend/` + `frontend/`) – use it as your template, but **your model must be trained on the new dataset below**.

Phase 0 – Prerequisites & Environment Setup

Install and verify each of these before starting:

Tool	Minimum version	Check with
Python	3.11	<code>python --version</code>
Node.js + npm	18	<code>node --version</code>
Git	any recent	<code>git --version</code>
A Kaggle account	–	https://www.kaggle.com
A GitHub account	–	https://github.com

```
mkdir house-price-project
cd house-price-project
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # macOS / Linux

pip install jupyter pandas numpy scikit-learn matplotlib seaborn
```

Create your project folder and a virtual environment:

Phase 1 – Get the Dataset

Dataset: **House Price** by Juhi Bhojani <https://www.kaggle.com/datasets/juhibhojani/house-price>

It contains real property listings from India (file `house_prices.csv`, ~187,000 rows) with columns such as `Title`, `Description`, `Amount(in rupees)`, `Price (in rupees)`, `location`, `Carpet Area`, `Status`, `Floor`, `Transaction`, `Furnishing`, `facing`, `overlooking`, `Society`, `Bathroom`, `Balcony`, `Car Parking`, `Ownership`, `Super Area`, `Dimensions`, `Plot Area`.

⚠ Always verify the actual columns yourself after downloading – run `df.columns` and `df.head()` first. Never trust a description over the real file.

Option A – Download manually: click **Download** on the dataset page, unzip, and place the CSV in `notebooks/data/`.

Option B – Kaggle CLI (recommended):

```
pip install kaggle
# Get your API token: Kaggle → Settings → API → "Create New Token"
# Place kaggle.json in C:\Users\<you>\.kaggle\ (Windows) or ~/.kaggle/ (macOS/Linux)
kaggle datasets download -d juhibhojani/house-price -p notebooks/data --unzip
```

Phase 2 – The Notebook: Train, Evaluate, Export

Create `notebooks/house_price_model.ipynb`. Your notebook must contain **all** of the following sections (use markdown headers so it reads like a report).

2.1 Load & Inspect

```
import pandas as pd

df = pd.read_csv("data/house_prices.csv")
df.shape           # how many rows/columns?
df.head()
df.info()          # dtypes, missing values
df.describe()
df.isna().mean().sort_values(ascending=False)  # % missing per column
```

Write a short markdown cell answering: How many rows? Which columns are numeric vs text? Which columns have the most missing values?

2.2 Exploratory Data Analysis (EDA)

Produce **at least 4 plots** and comment on each:

- Distribution of the target price (it will be heavily skewed – try a log scale).
- Price vs. carpet area (scatter).
- Average price by top-15 locations (bar chart).
- Price by furnishing status / number of bathrooms (box plots).

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df["price_clean"], log_scale=True)
plt.title("Price distribution (log scale)")
plt.show()
```

2.3 Cleaning & Feature Engineering

This dataset is **messy on purpose** – that is the main learning objective. Typical problems you must handle:

1. **Price is text.** Amount(in rupees) contains values like "42 Lac", "1.2 Cr", "Call for Price". Convert to a number (1 Lac = 100,000 ₹; 1 Cr = 10,000,000 ₹) and drop rows without a usable price.

```
def parse_amount(x):
    if not isinstance(x, str):
        return None
    x = x.strip().lower()
    try:
        if "lac" in x:
            return float(x.replace("lac", "").strip()) * 1e5
        if "cr" in x:
            return float(x.replace("cr", "").strip()) * 1e7
        return float(x.replace(",", ""))
    except ValueError:
        return None

df["price_clean"] = df["Amount(in rupees)"].apply(parse_amount)
df = df.dropna(subset=["price_clean"])
```

2. **Areas are text** – Carpet Area / Super Area look like "1200 sqft" or "140 sqm". Extract the number and normalise the unit to sqft (1 sqm ≈ 10.764 sqft).
3. **Floor** looks like "3 out of 10" → extract the floor number (handle "Ground", "Basement").
4. **Bathroom / Balcony / Car Parking** – convert to numeric, impute missing with median or 0.
5. **High-cardinality categoricals** – location and Society have thousands of values. Keep only the top-N (e.g. 50) locations and group the rest into "other" before one-hot encoding.

6. **Drop useless columns** – `Index`, `Title`, `Description`, `Dimensions`, and anything mostly empty.
7. **Remove outliers** – e.g. drop listings with absurd price-per-sqft (below the 1st or above the 99th percentile).

2.4 Build a Pipeline & Train

Use a scikit-learn `Pipeline` + `ColumnTransformer` so preprocessing is **bundled inside the exported model** (this makes the backend much simpler):

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression

numeric_features = ["carpet_area_sqft", "floor_num", "bathroom", "balcony"]
categorical_features = ["location_grouped", "Furnishing", "Transaction", "Ownership", "facing"]

preprocessor = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), numeric_features),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))]), categorical_features),
])

X = df[numeric_features + categorical_features]
y = df["price_clean"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = Pipeline([("prep", preprocessor),
                  ("reg", RandomForestRegressor(n_estimators=200, random_state=42))])
model.fit(X_train, y_train)
```

Train **at least 2 models** (e.g. `LinearRegression` as a baseline and `RandomForestRegressor` or `GradientBoostingRegressor`) and compare them.

Tip: because price is skewed, training on `np.log1p(y)` and inverting with `np.expml` at prediction time usually improves results noticeably. Try both and report the difference.

2.5 Evaluate

Report on the **test set** for every model:

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

pred = model.predict(X_test)
print("MAE :", mean_absolute_error(y_test, pred))
print("RMSE:", root_mean_squared_error(y_test, pred))
print("R² :", r2_score(y_test, pred))
```

Also include:

- A **predicted vs. actual** scatter plot.
- A comparison table of all models – pick a winner and justify it in one paragraph.
- (Bonus) 5-fold cross-validation with `cross_val_score`.

2.6 Export the Model

```
import joblib
joblib.dump(model, "house_price.pkl")

# Sanity check: reload and predict one sample
loaded = joblib.load("house_price.pkl")
sample = X_test.iloc[[0]]
print("Reloaded prediction:", loaded.predict(sample))
```

Also save the list of allowed locations (for the frontend dropdown):

```
import json
json.dump(sorted(df["location_grouped"].unique().tolist()), open("locations.json", "w"))
```

 **Version pinning:** a pickle only loads reliably with the same scikit-learn version. Note your version (`sklearn.__version__`) and pin it in the backend

Phase 3 — Backend (FastAPI)

Mirror the structure of the reference project:

```

backend/
├── app/
│   ├── main.py                # FastAPI app, CORS, model loaded at startup (lifespan)
│   ├── api/routes/prediction.py # GET /health, POST /predict
│   ├── core/config.py         # Settings from .env (pydantic-settings)
│   ├── schemas/prediction.py  # PredictionRequest / PredictionResponse
│   ├── services/
│   │   ├── preprocessing.py   # Turn a request into a one-row DataFrame
│   │   └── inference.py       # Load .pkl, run predict
│   └── utils/logging_config.py
├── models/house_price.pkl     # ← copied from your notebook
├── tests/test_prediction.py
├── requirements.txt
├── .env.example
└── Dockerfile

```

Steps:

1. `pip install fastapi "uvicorn[standard]" pydantic pydantic-settings pandas scikit-learn joblib pytest httpx` and freeze them into `requirements.txt` (pin scikit-learn to your notebook's version!).
2. Define the request schema to match **your model's input features**, e.g.:

```

class PredictionRequest(BaseModel):
    location: str
    carpet_area_sqft: float
    floor_num: int
    bathroom: int
    balcony: int
    furnishing: str      # "Furnished" | "Semi-Furnished" | "Unfurnished"
    transaction: str     # "New Property" | "Resale"
    ownership: str
    facing: str

```

3. In `services/preprocessing.py`, build a one-row `pandas.DataFrame` with **exactly the column names used in training** (and map unknown locations to `"other"`). Because you exported a full Pipeline, no manual encoding is needed – the pipeline does it.
4. Implement `POST /predict` returning `{"predicted_price": <float>}` and `GET /health` returning `{"status": "ok"}`.
5. Load the model **once at startup** (FastAPI lifespan), not on every request.
6. Add CORS middleware allowing `http://localhost:5173`.
7. Write at least 2 tests with `TestClient`: one happy path, one invalid input (expect 422).
8. Run and verify:

```

uvicorn app.main:app --reload
# open http://localhost:8000/docs and test /predict from Swagger UI

```

Phase 4 — Frontend (React + TypeScript + Vite)

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install react-router-dom
```

Structure (mirror the reference project):

```
frontend/src/
├── api/predictionClient.ts      # fetch wrapper, base URL from VITE API BASE URL
├── components/PredictionForm.tsx
├── pages/HomePage.tsx | ResultPage.tsx | NotFoundPage.tsx
├── types/prediction.ts        # TS types mirroring the backend schema
└── App.tsx                   # routes: / , /result , * (404)
```

Requirements:

1. `.env` with `VITE_API_BASE_URL=http://localhost:8000` (+ commit a `.env.example`).
2. The form must use proper input types: dropdown (`<select>`) for location / furnishing / transaction, numeric inputs for area, floor, bathrooms, balconies. Populate the location dropdown from your exported `locations.json`.
3. Client-side validation (required fields, area > 0) with friendly error messages.
4. Show a loading state while the request runs, an error message if the API fails, and the predicted price on the result page formatted nicely (e.g. ₹ 42.5 Lac).
5. Verify the full flow: backend on 8000, `npm run dev` on 5173, submit the form, see a real prediction.

Phase 5 — Publish on GitHub

1. Create `.gitignore` **before** the first commit. It must exclude: `.venv/`, `__pycache__/`, `node_modules/`, `dist/`, `.env`, `*.log`, and the **raw dataset CSV** (it is large – don't commit it; the README explains how to download it). The `.pkl` model may be committed if it is < 50 MB.
2. Initialise and commit:

```
git init
git add .
git commit -m "House price prediction: notebook, FastAPI backend, React frontend"
```

3. Create a **public** repository on GitHub (no need to initialise with a README), then:

```
git remote add origin https://github.com/<your-username>/house-price-app.git
git branch -M main
git push -u origin main
```

4. Write a root **README.md** (use this repo's root README as a model). It must include: overview, architecture diagram, tech stack, project structure, dataset link + download instructions, backend & frontend setup steps, environment variables tables, API reference with a curl example, model metrics (MAE / RMSE / R^2 of your chosen model), and screenshots of the running app.
5. Verify like a stranger: clone your repo into a fresh folder and follow **only your README**. If any step fails, fix the README.

Deliverables Checklist

- ☐ `notebooks/house_price_model.ipynb` – runs top-to-bottom without errors (Kernel → Restart & Run All), with EDA plots, cleaning, ≥ 2 models compared, test metrics, and model export.
- ☐ `backend/` – FastAPI app with `/health` + `/predict`, `.env.example`, pinned `requirements.txt`, passing `pytest`.
- ☐ `frontend/` – React form → result page, `.env.example`, `npm run build` succeeds.
- ☐ `models/house_price.pkl` served by the backend and produced by the notebook.
- ☐ Root `README.md` good enough for a stranger to run the whole project.
- ☐ Public GitHub repository with a clean history (no `node_modules`, no `.venv`, no `.env`, no raw CSV).
- ☐ End-to-end demo works: form → API → model → predicted price on screen.

Grading Rubric (100 pts)

Area	Points	What we look for
Data cleaning & feature engineering	25	Price/area parsing, outliers, high-cardinality handling, justified decisions
EDA	10	≥ 4 meaningful plots with written interpretation
Modeling & evaluation	20	≥ 2 models, proper train/test split, MAE/RMSE/R ² on test set, comparison & conclusion
Backend	15	Correct schema, pipeline-based inference, startup loading, CORS, tests pass
Frontend	15	Working form + validation, loading/error states, clean result display
GitHub & README	10	Reproducible setup, clean history, complete README
Code quality	5	Structure, naming, no dead code, no secrets committed

Common mistakes that lose points: committing `.env` or the dataset; a notebook that only runs in the author's original cell order; scikit-learn version mismatch between notebook and backend; hard-coding `http://localhost:8000` in frontend components instead of using the env variable; training metrics reported on the training set instead of the test set.

Good luck – build something you'd be proud to pin on your GitHub profile! 🚀